

1.Arithmetic Operations :

clc

close all

clear all

a = 33;

b = 8;

sum = a+b

difference = a-b

product = a*b

division = a/b

power = a ^ b

Workspace	
Name ^	Value
a	33
b	8
difference	25
division	4.1250
power	1.4064e+12
product	264
sum	41

Command Window	
sum =	
41	
difference =	
25	
product =	
264	
division =	
4.1250	
power =	
1.4064e+12	

2. Variables and Assignments :

clc

close all

clear all

x = 30

y = 40

x = 15

z = x + y

Workspace		Command Window	
Name	Value		
 x	15	x =	
 y	40		30
 z	55	y =	
			40
		x =	
			15
		z =	
			55
		>>	

3. Built-in Functions :

clc

close all

clear all

SquareRoot = sqrt(25)

AbsoluteValue = abs(-12)

```
SineFunction = sin(pi/6)
```

```
data = [2, 4, 6, 8, 10];
```

```
avg = mean(data)
```

Workspace	
Name	Value
avg	6
AbsoluteVal...	12
data	[2,4,6,8,10]
SineFunction	0.5000
SquareRoot	5

```
Command Window

SquareRoot =

    5

AbsoluteValue =

    12

SineFunction =

    0.5000

avg =

    6

>>
```

4. Trigonometric Functions :

```
clc
```

```
close all
```

```
clear all
```

```
theta = pi/6
```

```
sine_value = sin(theta)
```

```
cosine_value = cos(theta)
```

```
tangent_value = tan(theta)
```

Workspace	
Name	Value
cosine_value	0.8660
sine_value	0.5000
tangent_val...	0.5774
theta	0.5236

Command Window	
theta =	0.5236
sine_value =	0.5000
cosine_value =	0.8660
tangent_value =	0.5774
>>	

5. Inverse Trigonometric Functions :

clc

close all

clear all

InverseSine = asin(0.5)

InverseCosine = acos(0.5)

InverseTangent = atan(1)

Workspace		Command Window	
Name	Value		
InverseCosi...	1.0472	InverseSine =	0.5236
InverseSine	0.5236	InverseCosine =	1.0472
InverseTang...	0.7854	InverseTangent =	0.7854
		>>	

6. Hyperbolic Functions :

clc

close all

clear all

sinh_value = sinh(1)

cosh_value = cosh(1)

tanh_value = tanh(1)

Workspace		Command Window	
Name	Value		
cosh_value	1.5431	sinh_value =	1.1752
sinh_value	1.1752	cosh_value =	1.5431
tanh_value	0.7616	tanh_value =	0.7616
		>>	

7. Exponential and Logarithmic Functions :

```
clc
close all
clear all
exp_value = exp(1)
log_value = log(exp_value)
log10_value = log10(100)
```

Workspace	
Name	Value
exp_value	2.7183
log10_value	2
log_value	1

```
Command Window

exp_value =

    2.7183

log_value =

    1

log10_value =

    2

>>
```

8. Rounding Functions :

```
clc
close all
clear all
rounded_value = round(12.5)
floored_value = floor(5.8)
ceiled_value = ceil(4.1)
fixed_value = fix(11.9)
```

Workspace		Command Window	
Name	Value		
 ceiled_value	5	rounded_value =	
 fixed_value	11	13	
 floored_value	5	floored_value =	
 rounded_va...	13	5	
		ceiled_value =	
		5	
		fixed_value =	
		11	
		>>	

9. Modulus and Remainder Functions :

clc

close all

clear all

mod_value1 = mod(23, 5)

rem_value1 = rem(23, 5)

mod_value = mod(-23, 5)

rem_value2 = rem(-23, 5)

Workspace		Command Window	
Name	Value		
mod_value1	3	mod_value1 =	
mod_value2	2	3	
rem_value1	3	rem_value1 =	
rem_value2	-3	3	
		mod_value2 =	
		2	
		rem_value2 =	
		-3	
		>>	

10. Conditional Statements :

10.1- If Statement :

```
clc
close all
clear all
x = 10;
if x > 0
disp('x is positive');
end
```

Workspace		Command Window	
Name	Value		
x	10	x is positive	
		>>	

10.2- Else and Else-If Statements :

```
clc
```

```

close all
clear all
x = -10
if x > 0
    disp('x is positive')
elseif x < 0
    disp('x is negative')
else
    disp('x is zero')
end

```

Workspace	
Name	Value
 x	-10

```

Command Window

x =

    -10

x is negative
>>

```

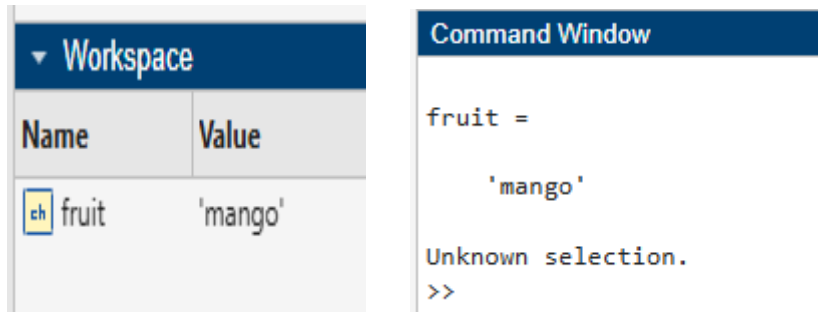
10.3- Switch Statement :

```

clc
close all
clear all
fruit = 'mango'
switch fruit
    case 'apple'
        disp('You selected an apple.')
    case 'orange'
        disp('You selected an orange.')
    otherwise

```

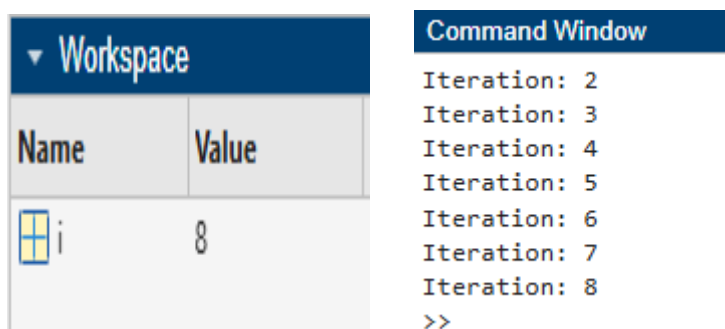
```
disp('Unknown selection.')  
end
```



11. Loops :

11.1- For Loop :

```
clc  
close all  
clear all  
for i = 2:8  
    disp(['Iteration: ', num2str(i)]);  
end
```



11.2 While Loop :

```
clc
```

```

close all
clear all
x = 3;
while x <= 9
    disp(['x is: ', num2str(x)]);
    x = x + 2;
end

```

Workspace		Command Window	
Name	Value		
x	11	<pre> x is: 3 x is: 5 x is: 7 x is: 9 >> </pre>	

12. Check If a Number is Positive or Negative :

```

clc
close all
clear all
number = -3
if number > 0
    disp('The number is positive.');
```

else

```

    disp('The number is negative.');
```

end

Workspace		Command Window	
Name	Value		
number	-3	<pre> number = -3 The number is negative. >> </pre>	

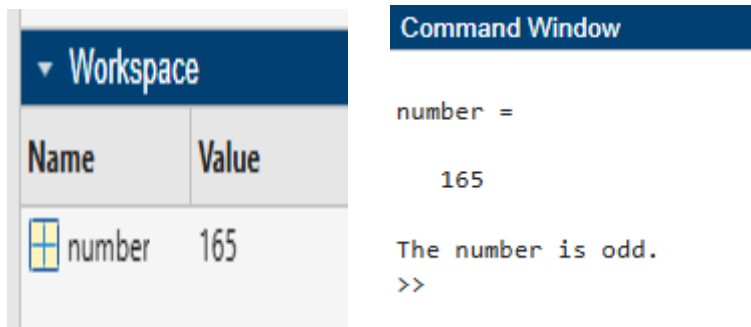
13. Determine if a Number is Even or Odd :

```
clc
close all
clear all
number = 165
if mod(number, 2) == 0
    disp('The number is even.');
```

else

```
    disp('The number is odd.');
```

end



14. Check If a Person is an Adult or a Minor :

```
clc
close all
clear all
age = 27
if age >= 18
    disp('You are an adult.');
```

else

```
    disp('You are a minor.');
```

end

Workspace		Command Window	
Name	Value		
age	27	age =	
		27	
		You are an adult.	
		>>	

15. Find the Larger of Two Numbers :

clc

close all

clear all

x = 25

y = 67

if x > y

 disp(['The larger number is ', num2str(x)]);

else

 disp(['The larger number is ', num2str(y)]);

end

Workspace		Command Window	
Name	Value		
x	25	x =	
		25	
y	67	y =	
		67	
		The larger number is 67	
		>>	

16. Check If a Student Passed or Failed :

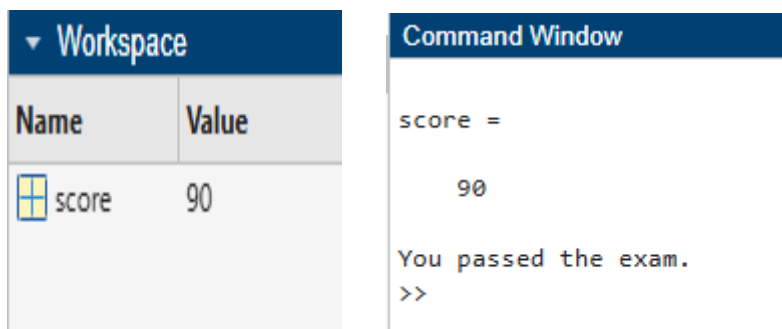
clc

```
close all  
clear all  
score = 90  
if score >= 60  
    disp('You passed the exam.');
```

else

```
    disp('You failed the exam.');
```

end



17. Determine If a Temperature is Above or Below Freezing :

```
clc  
clear all  
close all  
temperature = 10  
if temperature >= 0  
    disp("the temperature is above freezing.")  
else  
    disp("the temperature are below freezing")  
end
```

Workspace	
Name	Value
temperature	10

```
Command Window

temperature =

    10

the temperature is above freezing.
>>
```

18. Check If a Year is a Leap Year :

```
clc
clear all
close all
year = 2016
if mod(year, 4) == 0 && (mod(year, 100) ~= 0 || mod(year, 400) == 0)
    disp([num2str(year), ' is a leap year.']);
else
    disp([num2str(year), ' is not a leap year.']);
end
```

Workspace	
Name	Value
year	2016

```
Command Window

year =

    2016

2016 is a leap year.
>>
```

19. Compare String Lengths :

```
clc
clear all
close all
str1 = 'ENGINEERING';
```

```

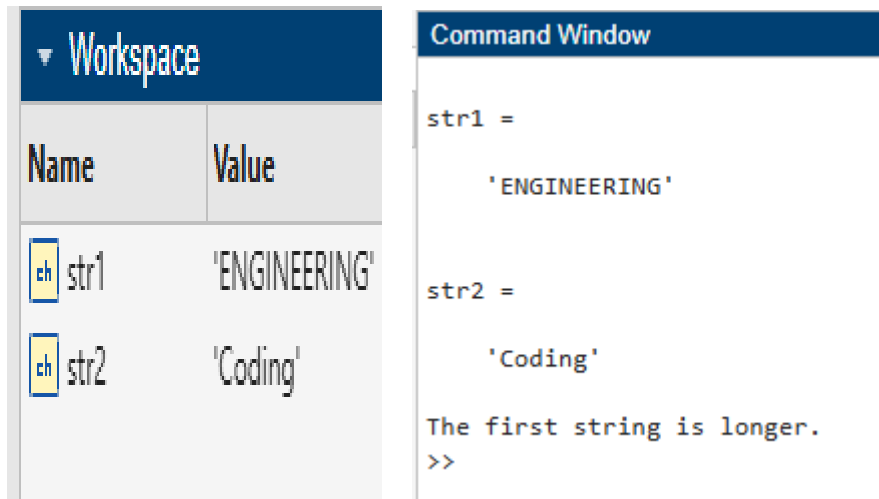
str2 = 'Coding';
if length(str1) > length(str2)
    disp('The first string is longer.');
```

else

```

    disp('The second string is longer or they are the same length.');
```

end



20. Check If a Number is Within a Specific Range :

```

clc
clear all
close all
number = 40
if number >= 10 && number <= 50
    disp('The number is within the range 10 to 50.');
```

else

```

    disp('The number is outside the range 10 to 50.');
```

end

Workspace		Command Window	
Name	Value		
number	40	<pre> number = 40 The number is within the range 10 to 50. >> </pre>	

21. Check If a Person is Eligible to Vote :

clc

clear all

close all

age = 24

citizen = true

if age >= 18 && citizen

 disp('You are eligible to vote.');

else

 disp('You are not eligible to vote.');

end

Workspace		Command Window	
Name	Value		
age	24	<pre> age = 24 citizen = logical 1 You are eligible to vote. >> </pre>	
citizen	1		

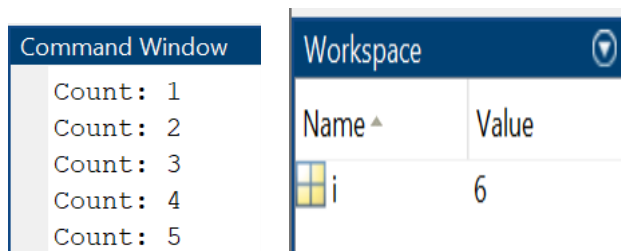
22. Count from 1 to 5 using a while loop :

```

clc
clear all
close all

i = 1;
while i <= 5
    disp(['Count: ', num2str(i)]);
    i = i + 1;
end

```



23. Sum numbers until the sum exceeds 100 :

```

clc
clear all
close all

sumValue = 0;
n = 1;
while sumValue <= 100
    sumValue = sumValue + n;
    n = n + 1;
end
disp(['Sum exceeded 100 at n = ', num2str(n-1)])

```

Command Window	
Sum exceeded 100 at n = 14	

Workspace	
Name ^	Value
n	15
sumValue	105

24. Print elements of a vector using a while loop :

```
clc
clear all
close all
vec = [2, 4, 6, 8, 10];
i = 1;
while i <= length(vec)
    disp(['Element ', num2str(i), ': ', num2str(vec(i))]);
    i = i + 1;
end
```

Workspace	
Name ^	Value
i	6
vec	[2,4,6,8,10]

Command Window	
Element 1: 2	
Element 2: 4	
Element 3: 6	
Element 4: 8	
Element 5: 10	

25. Continue loop until a random number is greater than 0.9 :

```
clc
clear all
close all
threshold = 0.9;
randomNum = rand;
while randomNum <= threshold
    disp(['Random number: ', num2str(randomNum)]);
```

```

    randomNum = rand;

end

disp('Exited the loop because random number > 0.9');

```

Workspace	
Name ^	Value
randomNu...	0.9058
threshold	0.9000

Command Window	
Random number: 0.81472	
Exited the loop because random number > 0.9	

26. Decrement a variable until it reaches zero :

```

clc

clear all

close all

x = 10;

while x > 0

    disp(['x is now: ', num2str(x)]);

    x = x - 1;

end

```

Workspace	
Name ^	Value
x	0

Command Window	
x is now: 10	
x is now: 9	
x is now: 8	
x is now: 7	
x is now: 6	
x is now: 5	
x is now: 4	
x is now: 3	
x is now: 2	
x is now: 1	

27. Multiply elements in an array until a condition is met :

```

clc

clear all

close all

arr = [1, 2, 3, 4, 5];

```

```

product = 1;

i = 1;

while product <= 50

    product = product * arr(i);

    disp(['Product: ', num2str(product)]);

    i = i + 1;

end

```

Workspace		Command Window
Name ^	Value	
arr	[1,2,3,4,5]	Product: 1
i	6	Product: 2
product	120	Product: 6
		Product: 24
		Product: 120

28. Find the first even number in an array :

```

clc

clear all

close all

arr = [5, 7, 11, 14, 19];

i = 1;

while mod(arr(i), 2) ~= 0

    i = i + 1;

end

disp(['First even number is: ', num2str(arr(i))]);

```

Workspace		Command Window
Name ^	Value	
arr	[5,7,11,14,19]	
i	4	First even number is: 14

29. Generate Fibonacci sequence until the value exceeds 100 :

```

clc
clear all
close all
a = 0;
b = 1;
while b <= 100
    disp(b);
    temp = b;
    b = a + b;
    a = temp;
end

```

Workspace	
Name ^	Value
a	89
b	144
temp	89

Command Window	
1	
1	
2	
3	
5	
8	
13	
21	
34	
55	
89	

30. Sum squares of numbers until the sum exceeds 200 :

```

clc
clear all
close all
sumSquares = 0;
n = 1;
while sumSquares <= 200
    sumSquares = sumSquares + n^2;

```

```
disp(['Sum of squares: ', num2str(sumSquares)]);
```

```
n = n + 1;
```

```
end
```

Workspace		Command Window	
Name ^	Value		
n	9	Sum of squares: 1	
sumSquares	204	Sum of squares: 5	
		Sum of squares: 14	
		Sum of squares: 30	
		Sum of squares: 55	
		Sum of squares: 91	
		Sum of squares: 140	
		Sum of squares: 204	

31. Find the smallest number divisible by both 3 and 5 :

```
clc
```

```
clear all
```

```
close all
```

```
n = 1;
```

```
while mod(n, 3) ~= 0 || mod(n, 5) ~= 0
```

```
    n = n + 1;
```

```
end
```

```
disp(['The smallest number divisible by both 3 and 5 is: ', num2str(n)]);
```

Workspace		Command Window	
Name ^	Value		
n	15	The smallest number divisible by both 3 and 5 is: 15	

32. Print numbers from 1 to 10 :

```
clc
```

```
close all
```

```
clear all
```

```
fprintf('Example 1: Print numbers from 1 to 10\n');
```

```

for i = 1:10
fprintf('%d\n', i);
end
fprintf('\n')

```

Workspace		Command Window	
Name	Value	Print numbers from 1 to 5	
i	5	1	
		2	
		3	
		4	
		5	
		>>	

33. Calculate the square of numbers from 1 to 5 :

```

clc
close all
clear all
fprintf('Example 2: Calculate the square of numbers from 1 to 5\n');
for i = 1:5
square = i^2;
fprintf('The square of %d is %d\n', i, square);
end
fprintf('\n');

```

Workspace		Command Window	
Name	Value	Calculate the square of numbers from 1 to 5	
i	5	The square of 1 is 1	
square	25	The square of 2 is 4	
		The square of 3 is 9	
		The square of 4 is 16	
		The square of 5 is 25	
		>>	

34. Calculate the sum of first 10 natural numbers :

```

clc
close all

```

```

clear all

fprintf('Example 3: Calculate the sum of first 10 natural numbers\n');

sum_result = 0;

for i = 1:10

    sum_result = sum_result + i;

end

fprintf('The sum of the first 10 natural numbers is %d\n', sum_result);

fprintf('\n');

```

Workspace		Command Window
Name	Value	Calculate the sum of first 10 natural numbers The sum of the first 10 natural numbers is 55 >>
i	10	
sum_result	55	

35. Display multiplication table of 5 :

```

clc

close all

clear all

fprintf('Example 4: Display multiplication table of 5\n');

n = 5;

for i = 1:10

    result = n * i;

    fprintf('%d x %d = %d\n', n, i, result);

end

```

```
fprintf('\n');
```

Workspace			
Name	Value	Size	Class
i	10	1×1	double
n	5	1×1	double
result	50	1×1	double

Command Window

Example 4: Display multiplication table of 5

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

```
>>
```

36. Print each element of an array :

```
clc
```

```
close all
```

```
clear all
```

```
fprintf('Example 5: Print each element of an array\n');
```

```
array = [10, 20, 30, 40, 50];
```

```
for i = 1:length(array)
```

```
    fprintf('Element %d: %d\n', i, array(i));
```

```
end
```

```
fprintf('\n');
```

Workspace	
Name	Value
array	[10,20,30,40,50]
i	5

Command Window

Print each element of an array

```
Element 1: 10
Element 2: 20
Element 3: 30
Element 4: 40
Element 5: 50
```

```
>>
```

37. Calculate the factorial of a number :

```
clc
close all
clear all
fprintf('Example 6: Calculate the factorial of a number\n');
n = 5;
factorial_result = 1;
for i = 1:n
    factorial_result = factorial_result * i;
end
fprintf('The factorial of %d is %d\n', n, factorial_result);

fprintf('\n');
```

Workspace					Command Window	
Name	↑	Value	Size	Class		
factorial_r...		120	1×1	double	Calculate the factorial of a number	
i		5	1×1	double	The factorial of 5 is 120	
n		5	1×1	double	>>	

38. Sum of even numbers from 1 to 20 :

```
clc
close all
clear all
fprintf('Example 7: Sum of even numbers from 1 to 20\n');
sum_result = 0;
for i = 2:2:20
    sum_result = sum_result + i;
end
```

```
fprintf('The sum of even numbers from 1 to 20 is %d\n', sum_result);
```

```
fprintf('\n');
```

Workspace					
Name	Value	Size	Class	Command Window	
i	20	1×1	double	Sum of even numbers from 1 to 10	
sum_result	110	1×1	double	The sum of even numbers from 1 to 10 is 30	
				>>	

39. Generate a sequence of odd numbers from 1 to 15

```
clc
```

```
close all
```

```
clear all
```

```
fprintf('Example 8: Generate a sequence of odd numbers from 1 to 15\n');
```

```
odd_numbers = [];
```

```
for i = 1:2:15
```

```
    odd_numbers = [odd_numbers, i];
```

```
end
```

```
disp('Odd numbers from 1 to 15:');
```

```
disp(odd_numbers);
```

```
fprintf('\n');
```

Workspace	
Name	Value
i	15
odd_numbers	[1,3,5,7,9,11,13,15]

Command Window	
Generate a sequence of odd numbers from 1 to 15	
Odd numbers from 1 to 15:	
1	3
5	7
9	11
13	15
>>	

40. Reverse a string using a for loop :

```
clc
close all
clear all
fprintf('Example 9: Reverse a string using a for loop\n');
str = 'ENGINEERING';
reversed_str = '';
for i = length(str):-1:1
    reversed_str = [reversed_str, str(i)];
end
fprintf('Reversed string: %s\n', reversed_str);
fprintf('\n');
```

Workspace	
Name	Value
i	1
reversed_str	'GNIREENIGNE'
str	'ENGINEERING'

Command Window	
Example 9: Reverse a string using a for loop	
Reversed string: GNIREENIGNE	
>>	

41. Calculate the cumulative sum of an array :

```
clc
```

```

close all

clear all

fprintf('Example 10: Calculate the cumulative sum of an array\n');

array = [1, 2, 3, 4, 5];

cumulative_sum = zeros(1, length(array));

for i = 1:length(array)

    cumulative_sum(i) = sum(array(1:i));

end

disp('Cumulative sum of the array:');

disp(cumulative_sum);

```

Workspace			
Name	Value	Size	Class
array	[1,2,3,4,5]	1×5	double
cumulativ...	[1,3,6,10,15]	1×5	double
i	5	1×1	double

Command Window

```

Example 10: Calculate the cumulative sum of an array
Cumulative sum of the array:
    1     3     6    10    15

>>

```

42. Arrays and Matrices :

42.1- One-Dimensional Array (Vector) :

```

clc

close all

clear all

row_vector = [1, 2, 3, 4, 5];

disp('Row vector:');

disp(row_vector);

col_vector = [1; 2; 3; 4; 5];

disp('Column vector:');

disp(col_vector);

first_element = row_vector(1); % Access the first element

```

```
last_element = col_vector(end); % Access the last element
disp(['First element: ', num2str(first_element)]);
disp(['Last element: ', num2str(last_element)]);
```

Workspace			
Name	Value	Size	Class
col_vector	[1;2;3;4;5]	5×1	double
first_elem...	1	1×1	double
last_elem...	5	1×1	double
row_vector	[1,2,3,4,5]	1×5	double

```
Command Window
Row vector:
    1    2    3    4    5

Column vector:
    1
    2
    3
    4
    5

First element: 1|
Last element: 5
>>
```

42.2 Two-Dimensional Array (Matrix)

```
clc
close all
clear all

matrix = [1, 2, 3; 4, 5, 6; 7, 8, 9];
disp('Matrix:');
disp(matrix);
element_2_3 = matrix(2, 3);
disp(['Element at (2,3): ', num2str(element_2_3)]);
transpose_matrix = matrix';
disp('Transposed Matrix:');
disp(transpose_matrix);
matrix_mult = matrix * transpose_matrix;
disp('Matrix multiplied by its transpose:');
disp(matrix_mult);
```

```

elementwise_mult = matrix .* matrix;

disp('Element-wise multiplication of the matrix:');

disp(elementwise_mult);

```

Workspace	
Name	Value
element_2_3	6
elementwise_mult	[1,4,9;16,25,36;49,64,81]
matrix	[1,2,3;4,5,6;7,8,9]
matrix_mult	[14,32,50;32,77,122;50,122,194]
transpose_matrix	[1,4,7;2,5,8;3,6,9]

Command Window

```

Matrix:
     1     2     3
     4     5     6
     7     8     9

Element at (2,3): 6
Transposed Matrix:
     1     4     7
     2     5     8
     3     6     9

Matrix multiplied by its transpose:
    14    32    50
    32    77   122
    50   122   194

Element-wise multiplication of the matrix:
     1     4     9
    16    25    36
    49    64    81

```

43. Cell Arrays :

```
clc
close all
clear all

cell_array = {1, 'text', [1, 2, 3]; rand(2,2), 42, 'MATLAB'};

disp('Cell Array:');

disp(cell_array);

first_element = cell_array{1, 1};

disp(['First element: ', num2str(first_element)]);

matrix_element = cell_array{1, 3};

disp('Matrix in cell array:');

disp(matrix_element);

cell_array{2, 2} = 'New Value';

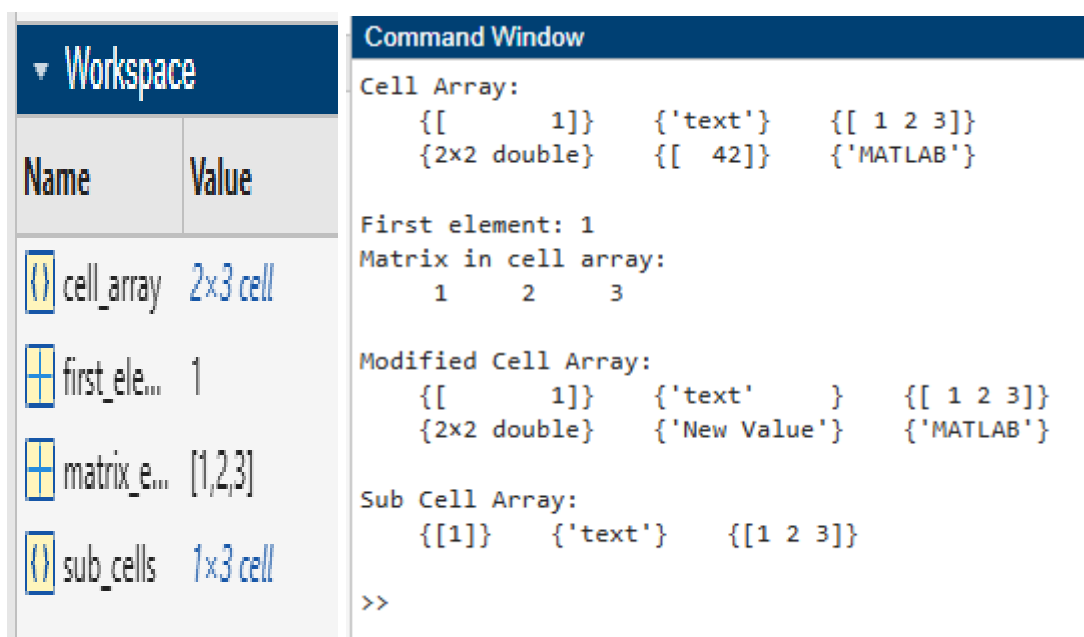
disp('Modified Cell Array:');

disp(cell_array);

sub_cells = cell_array(1, :);

disp('Sub Cell Array:');

disp(sub_cells);
```



The screenshot displays the MATLAB Command Window and Workspace. The Command Window shows the output of the script, and the Workspace shows the variables created.

Command Window

```
Cell Array:
    {[      1]}    {'text'}    {[ 1 2 3]}
    {2x2 double}    {[ 42]}    {'MATLAB'}

First element: 1
Matrix in cell array:
     1     2     3

Modified Cell Array:
    {[      1]}    {'text'    }    {[ 1 2 3]}
    {2x2 double}    {'New Value'}    {'MATLAB'}

Sub Cell Array:
    {[1]}    {'text'}    {[1 2 3]}

>>
```

Workspace

Name	Value
cell_array	2x3 cell
first_ele...	1
matrix_e...	[1,2,3]
sub_cells	1x3 cell

44. Structures :

```
clc
close all
clear all
student.name = 'John Doe';
student.age = 21;
student.scores = [90, 85, 88];
student.is_graduate = false;
disp('Structure:');
disp(student);
name = student.name;
disp(['Name: ', name]);
scores = student.scores;
disp('Scores:');
disp(scores);
student.age = 22;
disp('Modified Structure:');
disp(student);
student.major = 'Electrical Engineering';
disp('Structure with New Field:');
disp(student);
students(1).name = 'Alice';
students(1).age = 20;
students(1).major = 'Mathematics';
students(2).name = 'Bob';
students(2).age = 22;
students(2).major = 'Physics';
```

```
disp('Array of Structures:');  
disp(students);  
disp(['First student name: ', students(1).name]);  
disp(['Second student major: ', students(2).major]);
```

Workspace	
Name	Value
 name	'John Doe'
 scores	[90,85,88]
 student	1×1 struct
 students	1×2 struct

```

Command Window

Structure:
    name: 'John Doe'
    age: 21
    scores: [90 85 88]
    is_graduate: 0

Name: John Doe
Scores:
    90    85    88

Modified Structure:
    name: 'John Doe'
    age: 22
    scores: [90 85 88]
    is_graduate: 0

Structure with New Field:
    name: 'John Doe'
    age: 22
    scores: [90 85 88]
    is_graduate: 0
    major: 'Electrical Engineering'

Array of Structures:
    1x2 struct array with fields:

        name
        age
        major

First student name: Alice
Second student major: Physics
>>

```

45. Creating and Manipulating a Matrix :

```

Clc
close all
clear all

matrix = [1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12; 13, 14, 15, 16];

disp('Original Matrix:');

disp(matrix);

submatrix = matrix(1:2, 1:2);

disp('Submatrix (2x2 from top left):');

disp(submatrix);

matrix(3, 4) = 99;

disp('Matrix after modification:');

disp(matrix);

matrix = [matrix; 17, 18, 19, 20];

disp('Matrix after adding a row:');

disp(matrix);

det_matrix = det(matrix(1:3,1:3));

disp(['Determinant of the 3x3 matrix: ', num2str(det_matrix)]);

```

Workspace		Command Window	
Name	Value	Original Matrix:	
		<pre> 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 </pre>	
		Submatrix (2x2 from top left):	
		<pre> 1 2 5 6 </pre>	
		Matrix after modification:	
		<pre> 1 2 3 4 5 6 7 8 9 10 11 99 13 14 15 16 </pre>	
		Matrix after adding a row:	
		<pre> 1 2 3 4 5 6 7 8 9 10 11 99 13 14 15 16 17 18 19 20 </pre>	
		Determinant of the 3x3 matrix: -7.9936e-15	
		>>	
det_matrix	-7.9936e-15		
matrix	5x4 double		
submatrix	[1,2;5,6]		

46. Vector Operations :

```
clc
close all
clear all
row_vector = [1, 3, 5, 7, 9];
disp('Row Vector:');
disp(row_vector);
sum_vector = sum(row_vector);
disp(['Sum of elements: ', num2str(sum_vector)]);
squared_vector = row_vector.^2;
disp('Squared Vector:');
disp(squared_vector);
reversed_vector = flip(row_vector);
disp('Reversed Vector:');
disp(reversed_vector);
slice_vector = row_vector(2:4);
disp('Sliced Vector (elements 2 to 4):');
disp(slice_vector);
```

Workspace		Command Window	
Name	Value		
reversed_vec...	[9,7,5,3,1]	Row Vector:	
row_vector	[1,3,5,7,9]	1 3 5 7 9	
slice_vector	[3,5,7]	Sum of elements: 25	
squared_vect...	[1,9,25,49,81]	Squared Vector:	
sum_vector	25	1 9 25 49 81	
		Reversed Vector:	
		9 7 5 3 1	
		Sliced Vector (elements 2 to 4):	
		3 5 7	
		>>	

47. Matrix Arithmetic

```
clc
```

```
close all
```

```
clear all
```

```
A = [1, 2, 3; 4, 5, 6; 7, 8, 9];
```

```
B = [9, 8, 7; 6, 5, 4; 3, 2, 1];
```

```
C = A + B;
```

```
disp('Matrix Addition:');
```

```
disp(C);
```

```
D = A - B;
```

```
disp('Matrix Subtraction:');
```

```
disp(D);
```

```
E = A * B;
```

```
disp('Matrix Multiplication:');
```

```
disp(E);
```

```
F = A .* B;
```

```
disp('Element-wise Multiplication:');
```

```
disp(F);
```

```
G = inv(A + eye(3));
```

```
disp('Inverse of Matrix (A + I):');
```

```
disp(G);
```

Workspace		Command Window		
Name	Value	Matrix Subtraction:		
A	[1,2,3,4,5,6;7,8,9]	-8	-6	-4
B	[9,8,7,6,5,4;3,2,1]	-2	0	2
C	[10,10,10,10,10,10,10,10,10]	4	6	8
D	[-8,-6,-4;-2,0,2;4,6,8]	Matrix Multiplication:		
E	[30,24,18;84,69,54;138,114,90]	30	24	18
F	[9,16,21;24,25,24;21,16,9]	84	69	54
G	[-6,-2,3;-1,0,5000;-5.5511e-17,5,1,-2]	138	114	90
		Element-wise Multiplication:		
		9	16	21
		24	25	24
		21	16	9
		Inverse of Matrix (A + I):		
		-6.0000	-2.0000	3.0000
		-1.0000	0.5000	-0.0000
		5.0000	1.0000	-2.0000
		>>		

48. Reshaping and Transposing Matrices :

```

clc
close all
clear all
matrix = [1, 2, 3, 4, 5, 6; 7, 8, 9, 10, 11, 12];
disp('Original Matrix:');
disp(matrix);
reshaped_matrix = reshape(matrix, 3, 4);
disp('Reshaped Matrix (3x4):');
disp(reshaped_matrix);
transposed_matrix = reshaped_matrix';
disp('Transposed Matrix:');
disp(transposed_matrix);
column_vector = matrix(:);
disp('Matrix as a Column Vector:');
disp(column_vector);
flipped_matrix = flipud(matrix);
disp('Flipped Matrix (vertically):');

```

```
disp(flipped_matrix);
```

Workspace

Name	Value
 column_vector	12×1 double
 flipped_matrix	2×6 double
 matrix	2×6 double
 reshaped_matrix	3×4 double
 transposed_matrix	4×3 double

Command Window

Original Matrix:

1	2	3	4	5	6
7	8	9	10	11	12

Reshaped Matrix (3×4):

1	8	4	11
7	3	10	6
2	9	5	12

Transposed Matrix:

1	7	2
8	3	9
4	10	5
11	6	12

Matrix as a Column Vector:

1
7
2
8
3
9
4
10
5
11
6
12

Flipped Matrix (vertically):

7	8	9	10	11	12
1	2	3	4	5	6

>>

49. Working with Complex Numbers in Matrices :

```
clc
```

```
close all
```

```
clear all
```

```
complex_matrix = [1 + 2i, 2 - 3i; -1 + 4i, 3 - 2i];
```

```
disp('Complex Matrix:');
```

```
disp(complex_matrix);
```

```
real_part = real(complex_matrix);
```

```
disp('Real Part of the Matrix:');
```

```
disp(real_part);
```

```
imaginary_part = imag(complex_matrix);
```

```
disp('Imaginary Part of the Matrix:');
```

```

disp(imaginary_part);

magnitude_matrix = abs(complex_matrix);

disp('Magnitude of Complex Numbers:');

disp(magnitude_matrix);

phase_matrix = angle(complex_matrix);

disp('Phase (Angle) of Complex Numbers:');

disp(phase_matrix);

```

Workspace	
Name	Value
complex_matrix	[1 + 2i, 2 - 3i; -1 + 4i, 3 - 2i]
imaginary_part	[2, -3; 4, -2]
magnitude_matrix	[2.2361, 3.6056; 4.1231, 3.6056]
phase_matrix	[1.1071, -0.9828; 1.8158, -0.5880]
real_part	[1, 2; -1, 3]

Command Window

```

Complex Matrix:
    1.0000 + 2.0000i    2.0000 - 3.0000i
   -1.0000 + 4.0000i    3.0000 - 2.0000i

Real Part of the Matrix:
     1     2
    -1     3

Imaginary Part of the Matrix:
     2    -3
     4    -2

Magnitude of Complex Numbers:
    2.2361    3.6056
    4.1231    3.6056

Phase (Angle) of Complex Numbers:
    1.1071   -0.9828
    1.8158   -0.5880

>>

```

50. Creating and Accessing Cell Array Elements :

```
clc
```

```
clear all
```

```

close all

cell_array = {'Hello', 42, rand(2,2), [1, 2, 3]};

disp('Cell Array:');

disp(cell_array);

first_element = cell_array{1};

disp(['First Element: ', first_element]);

second_element = cell_array{2};

disp(['Second Element: ', num2str(second_element)]);

matrix_element = cell_array{3};

disp('Matrix in the Cell Array:');

disp(matrix_element);

vector_element = cell_array{4};

disp('Vector in the Cell Array:');

disp(vector_element);

```

Workspace		
Name	Value	Size
 cell_array	1×4 cell	1×4
 first_ele...	'Hello'	1×5
 matrix_e...	[0.8147,0.12...	2×2
 second_...	42	1×1
 vector_el...	[1,2,3]	1×3

```

Command Window

Cell Array:
    {'Hello'}    {[42]}    {2×2 double}    {[1 2 3]}

First Element: Hello
Second Element: 42
Matrix in the Cell Array:
    0.8147    0.1270
    0.9058    0.9134

Vector in the Cell Array:
     1     2     3

```

51. Modifying and Expanding Cell Arrays :

```

clc
clear all
close all

cell_array = {1, 'MATLAB', [3.14, 2.71]};
disp('Original Cell Array:');
disp(cell_array);

cell_array{2} = 'Updated Text';
disp('Modified Cell Array:');
disp(cell_array);

cell_array{4} = magic(3);
disp('Expanded Cell Array with New Element:');
disp(cell_array);

cell_array(1) = [];
disp('Cell Array after Removing the First Element:');
disp(cell_array);

another_cell_array = {true, 'New Data'};
combined_cell_array = [cell_array, another_cell_array];

```

```
disp('Combined Cell Array:');
```

```
disp(combined_cell_array);
```

▼ Workspace		
Name	Value	Size
 another_...	1×2 cell	1×2
 cell_array	1×3 cell	1×3
 combine...	1×5 cell	1×5

Command Window	
Original Cell Array:	
{[1]} {'MATLAB'} {[3.1400 2.7100]}	
Modified Cell Array:	
{[1]} {'Updated Text'} {[3.1400 2.7100]}	
Expanded Cell Array with New Element:	
{[1]} {'Updated Text'} {[3.1400 2.7100]} {3×3 double}	
Cell Array after Removing the First Element:	
{'Updated Text'} {[3.1400 2.7100]} {3×3 double}	
Combined Cell Array:	
{'Updated Text'} {[3.1400 2.7100]} {3×3 double} {[1]} {'New Data'}	

52. Using Cell Arrays with Loops :

```
clc
```

```
clear all
```

```
close all
```

```
cell_array = {1:5, 'text', eye(3), [10, 20, 30]};
```

```
for i = 1:length(cell_array)
```

```
    disp(['Element ', num2str(i), ':']);
```

```

        disp(cell_array{i});
    end
    for l = 1:length(cell_array)
        if isnumeric(cell_array{l})
            cell_array{l} = cell_array{l} * 2;
        elseif ischar(cell_array{l})
            cell_array{l} = upper(cell_array{l});
        end
    end
end
disp('Modified Cell Array:');
disp(cell_array);
concatenated_array = [cell_array{1}, cell_array{4}];
disp('Concatenated Array from Cell Array Elements:');
disp(concatenated_array);

```

▼ Workspace		
Name	Value	Size
 cell_array	1×4 cell	1×4
 concate...	[2,4,6,8,10,2...	1×8
 i	4	1×1

Command Window

Element 1:

1 2 3 4 5

Element 2:

text

Element 3:

1 0 0
0 1 0
0 0 1

Element 4:

10 20 30

Modified Cell Array:

{[2 4 6 8 10]} {'TEXT'} {3×3 double} {[20 40 60]}

Concatenated Array from Cell Array Elements:

2 4 6 8 10 20 40 60

53. Nested Cell Arrays :

clc

Clear all

Close all

Cell_array = {1:5, 'text', eye(3), [10, 20, 30]};

For I = 1:length(cell_array)

 Disp(['Element ', num2str(i), ':']);

 Disp(cell_array{i});

End

For I = 1:length(cell_array)

 If isnumeric(cell_array{i})

 Cell_array{i} = cell_array{i} * 2;

 Elseif ischar(cell_array{i})

```

        Cell_array{i} = upper(cell_array{i});
    End
End
Disp('Modified Cell Array:');
Disp(cell_array);
Concatenated_array = [cell_array{1}, cell_array{4}];
Disp('Concatenated Array from Cell Array Elements:');
Disp(concatenated_array);
Clc
Clear all
Close all
Nested_cell_array = {1, {2, 3, {'nested', 'cell', 'array'}}, 4};
Outer_element = nested_cell_array{2};
Disp('Outer Cell Element:');
Disp(outer_element);
Inner_element = nested_cell_array{2}{3};
Disp('Inner Cell Element:');
Disp(inner_element);
Innermost_element = nested_cell_array{2}{3}{2};
Disp(['Innermost Element: ', innermost_element]);
Nested_cell_array{2}{3}{2} = 'modified';
Disp('Nested Cell Array after Modification:');
Disp(nested_cell_array);

Function display_nested_cell(cell_array, indent)
    If nargin < 2
        Indent = '';
    End

```

```

For I = 1:length(cell_array)
    If iscell(cell_array{i})
        Disp([indent, 'Cell ', num2str(i), ':']);
        Display_nested_cell(cell_array{i}, [indent, ' ']);
    Else
        Disp([indent, 'Element ', num2str(i), ': ', num2str(cell_array{i})]);
    End
End

End

Disp('Display Nested Cell Array:');
Display_nested_cell(nested_cell_array);

```

Workspace		
Name	Value	Size
 inner_ele...	1×3 cell	1×3
 innermo...	'cell'	1×4
 nested_c...	1×3 cell	1×3
 outer_el...	1×3 cell	1×3

Command Window

Outer Cell Element:

`{[2]}      {[3]}      {1×3 cell}`

Inner Cell Element:

`{'nested'}      {'cell'}      {'array'}`

Innermost Element: cell

Nested Cell Array after Modification:

`{[1]}      {1×3 cell}      {[4]}`

Display Nested Cell Array:

Element 1: 1

Cell 2:

Element 1: 2

Element 2: 3

Cell 3:

Element 1: nested

Element 2: modified

Element 3: array

Element 3: 4

54. Cell Arrays for Storing Different-Length Data :

`Clc`

`Clear all`

`Close all`

`Data = {[1, 2, 3], [4, 5], [6, 7, 8, 9]};`

`Disp('Cell Array with Different-Length Vectors:');`

`Disp(data);`

▼ Workspace		
Name	Value	Size
 data	1×3 cell	1×3

Command Window

Cell Array with Different-Length Vectors:

```
{[1 2 3]}    {[4 5]}    {[6 7 8 9]}
```

```
>>
```